

Project Scope Statement

Heterogeneous OS Benchmarking for Real-Time Edge Control on Asymmetric Hardware

- **Course:** Operating Systems (Fall 2026)
- **Institution:** National Chung Cheng University (CCU)
- **Document Version:** 2.3.0 (Peer-Reviewed & Quantitative Execution Edition)
- **Document Status:** Active Execution (Milestone 1 Completed)

Table of Contents

- 1. Project Overview & Objectives
 - 1.1 Background & Motivation
 - 1.2 The Operating System Problem
 - 1.3 Project Objectives
- 2. High-Level System Components
- 3. Technical Environment & Tools
- 4. Scope Boundaries: In-Scope vs. Out-of-Scope
 - 4.1 In-Scope (What We Will Do)
 - 4.2 Out-of-Scope (What We Will NOT Do)
- 5. Key Project Deliverables
- 6. High-Level Work Breakdown Structure (High-Level WBS)
 - Team Roles & Lab Constraints
 - WBS Task Table (Phase 1 to Phase 7)
- 7. Project Schedule & Milestones
- 8. Acceptance Criteria (Definition of Success)
- 9. Assumptions, Constraints & Contingency Plans
 - 9.1 Assumptions
 - 9.2 Constraints & Risk Mitigations

1. Project Overview & Objectives

1.1 Background & Motivation

In smart manufacturing and industrial automation (such as CNC machining spindles and robotic joints), machinery relies on fast, repetitive control loops. For example, a motor controller must calculate and send corrective speed adjustments every 10 milliseconds (10 ms). If a control command arrives late by even a few milliseconds, the motor can vibrate violently, damaging the machine tool or ruining the manufactured workpiece.

Modern industrial systems are moving from **Predictive Maintenance (PdM)** to **Prescriptive Maintenance (PsM)**:

- **Predictive Maintenance (PdM):** An algorithm analyzes vibration data to warn an operator that a machine part is wearing down. Because when a technician reviews the alert, a few seconds of delay does not cause immediate damage (soft timing).
- **Prescriptive Maintenance (PsM):** The embedded controller autonomously takes corrective action within milliseconds (e.g., automatically slowing down or stopping a motor) before catastrophic failure occurs (hard real-time deadline).

1.2 The Operating System Problem

Executing closed-loop Prescriptive Maintenance reveals an operating system dilemma:

1. **Why Linux alone struggles:** General-Purpose Operating Systems (GPOS) like Linux provide rich software libraries for heavy math (like Fast Fourier Transform vibration analytics and neural network inference). However, Linux is designed to be fair to all running programs. When background tasks run, Linux may pause the control program to give CPU time to other apps or to manage memory. These unpredictable pauses can exceed 10 to 50 milliseconds, causing missed deadlines.
2. **Why a Microcontroller RTOS alone is not enough:** A Real-Time Operating System (RTOS) like Zephyr guarantees that time-critical tasks run on time without random pauses. However, microcontrollers have limited CPU power and small memory, making them incapable of running heavy vibration analytics.
3. **The Proposed Dual-OS Solution:** We investigate a **Heterogeneous Dual-OS Architecture** on a single dual-chip board (the **Arduino UNO Q**) leveraging a **Multi-Rate Asymmetric Execution Model**:
 - **Cognitive Plane on Linux Processor (The Analytical Brain):** Processes 1024-sample vibration frames (~85.3 ms of physical signal at 12 kHz) using FFT and a 1D-CNN classifier at an analytical cadence of ~80–100 ms. When an updated bearing health assessment is produced, Linux deposits the new speed derating recommendation into a shared memory mailbox.
 - **Reflex Plane on Zephyr Microcontroller (The Fast Muscle):** Executes the time-critical motor actuation loop at a strict, deterministic **10 ms interval (100 Hz)** with zero jitter. Zephyr reads the latest active setpoint from shared memory asynchronously without blocking or waiting for Linux.
 - **Inter-Processor Communication (IPC):** Lock-free shared memory channel that completely decouples the high-frequency control loop from the variable-duration AI inference pipeline.

1.3 Project Objectives

- **Objective 1 (Architecture Setup):** Establish a functional dual-OS communication pipeline between Linux and Zephyr RTOS on the Arduino UNO Q.
- **Objective 2 (Benchmarking Execution):** Run a periodic 10 ms control loop for **10,000 continuous cycles** across three configurations:
 - Configuration A: Monolithic Linux under standard Completely Fair Scheduling (CFS).
 - Configuration B: Monolithic Linux under POSIX Real-Time Priority Scheduling (`SCHED_FIFO` , priority 90) on the standard Linux kernel.
 - Configuration C: Heterogeneous Dual-OS (Linux Cognitive Plane + Zephyr RTOS Reflex Plane).
- **Objective 3 (Stress & Latency Evaluation):** Measure how each configuration behaves under escalating background workload stress (0%, 50%, 100% CPU/memory/IO stress using `stress-ng`) and quantify the **Deadline Miss Ratio (DMR)** and **Worst-Case Latency**.
- **Objective 4 (Deliverables):** Produce clear comparison charts, presentation slides, and an academic project report explaining the OS trade-offs.

2. High-Level System Components

Rather than focusing on low-level code implementation details at this proposal stage, the project is structured into four conceptual components:

+-----+	
	ARDUINO UNO Q HARDWARE
	[COMPONENT 1: Linux Analytics Subsystem]
	- Runs on the quad-core application processor (Qualcomm Cortex-A53)
	- Simulates vibration signal processing and health evaluations
	- Tracks operating system interruptions (context switches and memory stalls)
	[COMPONENT 3: Inter-Processor Communication (IPC) Bridge]
	- Fast message-passing channel between both processors using shared memory
	[COMPONENT 2: Zephyr Real-Time Control Subsystem]
	- Runs on the dedicated microcontroller (STM32 Cortex-M33)
	- Executes the 10 ms motor control loop with guaranteed real-time priority
	- Accurately measures cycle time using hardware timers
	[COMPONENT 4: Benchmarking & Measurement Harness]
	- Injects synthetic background stress to test system limits
	- Records latency data across 10,000 cycles and outputs CSV files for graphing
+-----+	

- Component 1 (Linux Analytics Subsystem):** Executes the heavy analytical workload (1024-point vibration FFT + 1D-CNN condition inference). Based on the detected bearing condition (Normal, Ball Fault, Inner Race Fault, Outer Race Fault), a **deterministic rule-based Look-Up Table (LUT / if-else policy)** calculates the prescribed motor speed derating setpoint (100%, 85%, 70%, or 60%) to relieve mechanical stress.
- Component 2 (Zephyr Real-Time Control Subsystem):** Runs the time-sensitive motor control task at a fixed 10 ms interval. Because it runs on a dedicated real-time microcontroller, it is isolated from Linux background activity.
- Component 3 (Inter-Processor Communication Bridge):** A shared mailbox where Linux deposits speed recommendations, and the Zephyr microcontroller reads them safely without waiting or freezing.
- Component 4 (Benchmarking & Measurement Harness):** Automated test scripts that run the 10,000-cycle experiment under different stress levels and record timing logs for analysis.

3. Technical Environment & Tools

- Target Hardware:** Arduino UNO Q
 - Processor 1: Qualcomm QRB2210 Quad-Core ARM Cortex-A53 (runs Linux).
 - Processor 2: STM32U585 ARM Cortex-M33 (runs Zephyr RTOS).
- Operating Systems:**
 - Embedded Linux (Debian-based) for the application processor.
 - Zephyr RTOS (v3.x) for the real-time microcontroller.
- Development Tools & Utilities:**
 - Programming Language: C / Shell scripting / Python.
 - Build System & Toolchain: CMake, west (Zephyr meta-tool), and **Ninja** (a high-speed, parallel build execution backend required by Zephyr RTOS to compile C/C++ firmware rapidly), GCC ARM Embedded toolchain (arm-none-eabi-gcc).
 - Local RTOS Emulation & Testing: Zephyr native POSIX simulator (native_sim) or QEMU ARM Cortex-M emulator (qemu_cortex_m3), enabling the remote teammate (Akash) to write, compile, and functionally verify Zephyr periodic timer apps locally without requiring the physical board.
 - System Stress Generator: stress-ng (a standard Linux tool used to simulate high CPU and memory workload).
 - Operating System Metrics: Standard OS timing functions (clock_nanosleep , clock_gettime) and resource usage statistics (getrusage) to record task interruptions.
 - Data Analysis & Plotting: Python (matplotlib / numpy / pandas) to generate latency distribution graphs, CDF curves, and comparison box plots.

- **Benchmark Dataset Specification:**

- **Name:** Case Western Reserve University (CWRU) Bearing Data Center Seeded Fault Dataset.
- **Source / Download URL:** Official portal at <https://engineering.case.edu/bearingdatacenter> (Mirror/clean packages on Kaggle: `brnithish/cwr-bearing-dataset` or GitHub: `hasan-kamal/Bearing-Fault-Detection-CWRU`).
- **Selected Subset:** 12k Drive End Bearing Fault Data sampled at 12,000 Hz under 1–3 HP motor loads.
- **Target Classes:** 4 operational conditions — Normal Baseline (`97.mat`), Ball Fault (`118.mat`), Inner Race Fault (`105.mat`), Outer Race Fault (`130.mat`).
- **Repository Location:** Downloaded into `data/raw/cwr/` and preprocessed into fixed 1024-sample windows in `data/processed/` for FFT analysis and model inference.

4. Scope Boundaries: In-Scope vs. Out-of-Scope

Defining clear boundaries is critical to keeping the project focused and achievable within one academic semester:

4.1 In-Scope (What We Will Do)

- Configure the Arduino UNO Q board to run both Linux (Debian) and Zephyr RTOS.
- Implement realistic vibration analytical workload using CWRU bearing dataset playback (1024-point frames, FFT, TFLite 1D-CNN inference) coupled with a periodic 10 ms control loop.
- Implement an inter-processor message exchange between Linux and Zephyr using shared memory.
- Benchmark 10,000 continuous cycles for:
 - Standard Linux (CFS).
 - Real-Time Linux (`SCHED_FIFO` , priority 90) on the standard Linux kernel.
 - Dual-OS (Linux Cognitive Plane + Zephyr RTOS Reflex Plane).
- Compare the three configurations under no load vs. heavy background stress (`stress-ng`).
- Measure average latency, worst-case latency, jitter standard deviation, and number of missed deadlines.
- Produce academic reports and presentations matching course requirements.

4.2 Out-of-Scope (What We Will NOT Do)

- **No Physical Motor Rig:** No real motors or wiring are built. We replay recorded CWRU vibration data and use timers to focus purely on OS behavior.
- **No Novel Machine Learning / AI Research:** We do not invent new AI architectures. A standard lightweight 1D-CNN is used strictly to represent a realistic edge analytical workload.
- **No Complex Numerical Optimization for Prescriptions:** The Prescriptive Maintenance (PsM) decision is implemented as a fast, deterministic rule-based Look-Up Table (LUT / `if-else` derating policy) that maps detected fault classes to speed setpoints (100%, 85%, 70%, 60%). Complex multi-objective quadratic optimization or enterprise factory scheduling solvers are intentionally excluded to keep the focus squarely on Operating System scheduling latency, page faults, and inter-processor communication determinism.
- **No Graphical User Interface (GUI):** Tests will run from the command line (CLI) and generate log files; no complex desktop or web UI will be created.
- **No Cloud or Internet Connectivity:** All experiments run locally on the board. Wi-Fi and Cloud networking are excluded to prevent network lag from distorting OS measurement data.

5. Key Project Deliverables

1. Source Code & Scripts:

- Linux analytics program and telemetry collector.
- Zephyr RTOS microcontroller firmware.
- Inter-processor communication interface.

- Automated benchmark test scripts.

2. **Experimental Data & Visualizations:**

- Raw CSV log files recording 10,000 cycles for each test setup.
- Latency distribution graphs (box plots and Cumulative Distribution Function curves) comparing the three configurations.

3. **Course Deliverables & Academic Reports:**

- **Milestone 1 (Sep 29):** 2-Page Project Proposal & Abstract + 5-10 min in-class presentation. ✔ **COMPLETED**
- **Milestone 2 (Oct 20):** Formal Written Progress Report describing environment setup and preliminary tests. ⌚ **3 Weeks Remaining**
- **Milestone 3 (Nov 10):** 25-Minute In-Class Presentation and live/recorded demo. ⌚ **6 Weeks Remaining**
- **Milestone 4 (Dec 29):** Final Written Report (up to 15 pages in IEEE format), codebase, and role breakdown. ⌚ **13 Weeks Remaining**

6. High-Level Work Breakdown Structure (High-Level WBS)

Team Roles

The board (Arduino UNO Q) is located in Huy's lab. Akash works remotely from a separate lab and has no physical board access. Work is divided accordingly:

- **Huy (Board & Linux Lead):** All tasks requiring physical board access - SSH, flashing, running benchmarks, IPC integration testing.
- **Akash (Model, Analysis & Documentation Lead):** All laptop-only tasks - model training, Zephyr firmware development (compile only), data analysis, plotting, and report drafting.

WBS Table

WBS	Phase / Task	Owner	Board?	Status
P1	Phase 1: Project Initiation & Proposal (Wk 1-3, Sep 14 - Sep 29) — [Milestone 1]			✔ Done
P1.1	Identify core problem: GPOS vs RTOS trade-off for PsM	Both	-	✔
P1.2	Select hardware platform (Arduino UNO Q: A53 + M33)	Both	-	✔
P1.3	Define 3 benchmark configs (A: CFS, B: RT-FIFO, C: Dual-OS)	Both	-	✔
P1.4	Submit 2-page proposal and deliver in-class presentation	Both	-	✔
P2	Phase 2: Linux Baseline Benchmarking & Bring-up (Wk 4-6, Sep 30 - Oct 20) — [Supports M2]			🚀 Active
	Goal: Collect Config A & B results for M2 Progress Report			
P2.1	Connect to Linux processor (Cortex-A53) via SSH / serial	Huy 🔧	Yes	
P2.2	Install <code>stress-ng</code> , <code>build-essential</code> on board	Huy 🔧	Yes	
P2.3	Ingest CWRU bearing vibration dataset (<code>data/raw/cwru/</code>) onto board filesystem	Huy 🔧	Yes	
P2.4	Write 10 ms periodic C loop (read vibration frame, FFT, log CSV)	Huy 🔧	Yes	
P2.5	Run Config A: standard Linux CFS scheduling	Huy 🔧	Yes	
P2.6	Run Config B: RT-FIFO scheduling (<code>sudo chrt -f 90</code>)	Huy 🔧	Yes	
P2.7	Evaluate both under idle and stress (<code>stress-ng --cpu 4 --vm 1</code>)	Huy 🔧	Yes	

WBS	Phase / Task	Owner	Board?	Status
P2.8	Write shell scripts for automated stress test scenarios	Akash 	No	
P2.9	Draft and submit M2 Progress Report (Due Oct 20)	Both	-	
P3	Phase 3: Offline Model Training & Edge Deployment (Wk 4-6, Oct 1 - Oct 20, Parallel Track)			
	<i>Goal: Train 1D-CNN on CWRU dataset and export TFLite INT8 model</i>			
P3.1	Download CWRU raw data into <code>data/raw/cwru/</code> , segment into 1024-sample windows	Akash 	No	
P3.2	Train 1D-CNN fault classifier (normal / inner / ball / outer race)	Akash 	No	
P3.3	Quantize model to TFLite INT8 (<code>.tflite</code>)	Akash 	No	
P3.4	Receive <code>.tflite</code> from Akash, deploy to board via <code>scp</code> / <code>sftp</code>	Huy 	Yes	
P3.5	Integrate TFLite C/C++ runtime into control loop (FFT -> inference -> prescription)	Huy 	Yes	
P4	Phase 4: Zephyr RTOS Bring-up & Real-Time Timer Setup (Wk 5-7, Oct 7 - Oct 27)			
	<i>Goal: Set up Zephyr toolchain (west + CMake + Ninja) and establish zero-jitter baseline</i>			
P4.1	Install Zephyr SDK, CMake, Ninja, <code>west</code> on laptop; test with <code>native_sim</code>	Akash 	No	
P4.2	Configure BSP, device tree, pin mux for STM32U585 Cortex-M33	Akash 	No	
P4.3	Develop Zephyr 10 ms timer app (<code>k_timer</code>); verify logic locally in <code>native_sim</code>	Akash 	No	
P4.4	Receive binary from Akash, flash to MCU, verify jitter over UART	Huy 	Yes	
P4.5	Confirm zero-jitter RTOS baseline before IPC connection	Huy 	Yes	
P4.6	Remote debugging via screen-share (Akash adjusts code, Huy re-flashes)	Both	Yes	
P5	Phase 5: Dual-OS Communication & Live Demo (Wk 7-9, Oct 26 - Nov 10) — [Supports M3]			
	<i>Goal: Complete Config C and deliver M3 Presentation & Demo</i>			
P5.1	Write Zephyr-side IPC: read prescription from shared memory, apply in actuation loop	Akash 	No	
P5.2	Write Linux-side IPC: serialize FFT + TFLite output, write to shared memory	Huy 	Yes	
P5.3	Deploy both sides on board, run end-to-end integration tests	Huy 	Yes	
P5.4	Inject <code>stress-ng</code> on Linux, verify Zephyr actuation is immune	Huy 	Yes	

WBS	Phase / Task	Owner	Board?	Status
P5.5	Prepare 25-minute presentation slides (joint slide deck)	Both	-	
P5.6	Co-present and demo for M3 (Due Nov 10)	Both	Yes	
P6	Phase 6: Full-Scale Benchmarking & Statistical Evaluation (Wk 10-12, Nov 11 - Dec 6)			
	<i>Goal: Collect 10,000-cycle datasets across 9 test matrices for M4</i>			
P6.1	Execute 10,000-cycle runs across 9 matrices (3 configs x 3 stress levels), export CSVs	Huy 🛠️	Yes	
P6.2	Generate visualizations: CDF curves, latency box plots, jitter distribution	Akash 🖥️	No	
P6.3	Calculate KPIs: DMR, 99.9th percentile worst-case latency, jitter std dev	Akash 🖥️	No	
P7	Phase 7: IEEE Final Report & Codebase Packaging (Wk 13-16, Dec 7 - Dec 29) — [Supports M4]			
	<i>Goal: Submit 15-page IEEE report and complete code repository</i>			
P7.1	Write sections: hardware setup, Linux experiments (A & B), IPC implementation	Huy 🛠️	No	
P7.2	Write sections: model design, Zephyr RTOS theory, statistical analysis, conclusions	Akash 🖥️	No	
P7.3	Cross-review each other's sections and unify writing style	Both	-	
P7.4	Clean and document code repository with test scripts and reproduction steps	Both	-	
P7.5	Document individual contributions, submit final package M4 (Due Dec 29)	Both	-	

Risk Mitigation Note: Because Phase 2 (Config A vs Config B) produces valid comparative OS data by Oct 20 (M2), the project already has tangible academic deliverables even while Zephyr/IPC integration is underway for M3.

7. Project Schedule & Milestones

The project milestones are aligned with the official deadlines established in the CCU Operating System course syllabus:

Milestone	Target Date	Status	Time Window	Contributing WBS Phases	Description & Major Deliverables	Primary Focus
M1	Sep 29	 COMPLETED	Weeks 1 - 3	Phase 1 (P1)	Project Proposal & Abstract: Submitted 2-page proposal and presented in class.	Problem definition, scope, and dual-OS concept
M2	Oct 20	 UPCOMING	Weeks 4 - 6 (3 wks left)	Phase 2 (P2) & Phase 3 (P3)	Progress Report: Submit formal written report detailing setup and preliminary results.	Platform bring-up, baseline benchmarks (Config A & B), and offline model

Milestone	Target Date	Status	Time Window	Contributing WBS Phases	Description & Major Deliverables	Primary Focus
M3	Nov 10	🕒 UPCOMING	Weeks 7 - 9 (6 wks left)	Phase 4 (P4) & Phase 5 (P5)	Class Presentation & Demo: Deliver 25-minute presentation and live/recorded demo.	Dual-OS integration (Config C), model inference, and stress evaluation
M4	Dec 29	🕒 FINAL	Weeks 10 - 16 (13 wks left)	Phase 6 (P6) & Phase 7 (P7)	Final Report Due: Submit complete 15-page report in IEEE format, code repository, and roles.	In-depth OS analysis, complete 10k-cycle data, and documentation

8. Acceptance Criteria (Definition of Success)

The project will be considered successful when the following quantitative engineering benchmarks and academic standards are fulfilled:

- Benchmark Execution Stability:** All 10,000 continuous control cycles execute reliably to completion without application crash, kernel panic, or buffer overflow across all 9 experimental test runs (3 Configurations × 3 Stress Levels).
- Quantitative Real-Time Metrics & Thresholds:**
 - Nominal Cycle Period (T_{nom}):** Exactly 10.0 ms (100 Hz).
 - Deadline Miss Definition:** Any cycle period or turnaround latency exceeding 12.0 ms (a +20% jitter threshold) is officially flagged as a **Deadline Miss**. Delays > 50.0 ms are flagged as **Severe System Starvation**.
 - Empirical Target KPIs:**
 - Configuration C (Dual-OS):** Must achieve a Deadline Miss Ratio of $DMR = 0.00\%$ under 100% Linux stress (`stress-ng --cpu 4 --vm 1`), with cycle jitter standard deviation $\sigma < 0.10$ ms, proving absolute temporal isolation.
 - Configurations A & B (Linux CFS & `SCHED_FIFO`):** Must capture statistically significant latency degradation under heavy load ($DMR > 0\%$ and max latency spikes > 20 ms), providing empirical validation of GPOS CPU quota throttling and memory page-fault stalls.
- Academic Deliverable Standards:**
 - All four course milestones (M1, M2, M3, M4) submitted strictly by the official deadlines.
 - Individual teammate contributions (Huy vs. Akash) clearly documented to satisfy the syllabus peer-evaluation criteria.
 - Publication-grade final IEEE report (up to 15 pages) accompanied by fully documented, reproducible scripts in the repository.

9. Assumptions, Constraints & Contingency Plans

9.1 Assumptions

- The Arduino UNO Q hardware board remains functional and accessible in Huy’s laboratory.
- Synthetic stress generated by `stress-ng` provides an industry-standard, reproducible simulation of high edge computing workloads.
- The CWRU bearing dataset provides a recognized, verifiable baseline for vibration frequency spectra.

9.2 Constraints & Risk Mitigations

- Single Hardware Board Constraint:** Only Huy has physical access to the board.

- *Mitigation:* Akash utilizes Zephyr's host emulator (`native_sim`) and QEMU to write, build, and test Zephyr application logic locally before passing compiled binaries to Huy for hardware deployment.
- **Hardware Corruption / Kernel Crash Risk:** Intensive stress testing (`stress-ng`) and direct hardware register access can cause OS crashes or filesystem corruption.
 - *Mitigation:* A golden backup image of the Debian filesystem is maintained on cloud storage, allowing full board re-flashing within 15 minutes.
- **Contingency Schedule Buffer:** If dual-OS inter-processor communication setup takes longer than anticipated, Phase 2 (Config A vs Config B on Linux) provides complete, valid comparative operating systems data for Milestone 2 (Oct 20), guaranteeing that the team never misses an academic deadline.